Lezione 5 · Trovare l’errore, e un quadro tutto tuo

Giovedì 22 ottobre 2026 · 90 minuti · 45 minuti di tastiera

Cosa fa questa lezione

Traguardo 1: sanno cosa fare quando compare il testo rosso, invece di alzare la mano e dire «non va».

Traguardo 2: ognuno finisce l’ora con un quadro suo su uno sfondo già pronto — cielo, prato e sole — fatto mettendo insieme pezzi che non ha dovuto scrivere da zero.

La quarta lezione ha detto una cosa chiara: il collo di bottiglia non è capire, è **battere sulla tastiera**. Venti minuti per un quadrato, e metà del tempo se ne va in errori di battitura che nessuno sa leggere. Quindi oggi si cambia metodo, e il cambiamento vale per tutte le lezioni che restano:

> **Non si parte mai più da un file vuoto.**
>
> Aprono un file che **già funziona** e lo cambiano. Battono un quinto, vedono un risultato ogni trenta secondi, e imparano di più: per cambiare un numero bisogna prima capire cosa fa.
>
> Scrivere da zero torna più avanti, quando la tastiera non farà più male.

Niente cicli oggi. Nel file `treppe.py` ci sono cinque gradini scritti cinque volte uguali, e in cima c’è scritto di ricordarselo. È l’esca per la settimana prossima.

Il materiale

Cosa	Quanto	Nota
Portatili con Thonny	11 + 1	
I nove file di esempio, **già sulle macchine**	—	Nella cartella `python_kurs`, copiati il mercoledì
I sei file rotti, **in una sottocartella** `fehler`	—	Separati, se no li aprono per sbaglio
Proiettore	1	
Quaderni e foglietti	11	Dalla scatola, come sempre

> **La preparazione del mercoledì, venti minuti**
>
> Su ogni macchina, dentro `python_kurs`:
>
> - i nove file di esempio, sciolti;
> - una cartella `fehler` coi sei file rotti dentro.
>
> Chiavetta e copia-incolla, macchina per macchina. È l’unica cosa che se salta fa saltare la lezione, perché oggi tutto parte da file che devono essere già lì.

I novanta minuti

Minuti	Fase	
0–5	Apertura	Cosa si fa oggi. Due frasi
5–20	**Fehlerjagd**	Sei file rotti, tre domande
20–25	I file di esempio	Li apri uno per uno al proiettore, tre secondi ciascuno
25–30	Il triangolo, due modi	L’unico pezzo di teoria della giornata
30–35	Lo sfondo	Dove va il loro pezzo

35–80	Officina	Ognuno costruisce il suo quadro
80–88	Galleria	Proietti tre o quattro schermi
88–90	Quaderno	Una riga

Fase 1 · Apertura (0–5)

Due frasi, niente di più:

> »Heute zwei Sachen. Erst lernen wir, wie man einen Fehler findet. Dann baut jeder sein eigenes Bild — und ihr müsst fast nichts tippen.«

Poi apri `hintergrund.py` al proiettore e lo esegui: cielo, prato, sole. »Da drauf kommt euer Bild.«

Fase 2 · Fehlerjagd (5–20)

Quindici minuti, ed è la parte che ti fa guadagnare tempo per tutto il resto del trimestre.

Le tre domande

Scrivile alla lavagna e lasciale lì tutta l'ora. Sono il gesto da imparare, non una spiegazione.

> **1. Welche Zeile?** — il numero è scritto, basta leggerlo.
>
> **2. Was steht da?** — vai su quella riga e leggila ad alta voce.
>
> **3. Was ist anders als auf dem Spickzettel?** — il confronto, non la memoria.

Minuti 5–8: il primo insieme

`fehler_1.py` al proiettore. Premi Play, e **leggi il messaggio rosso ad alta voce**, tutto, anche se è in inglese. Poi fai tu le tre domande, una per volta, e risolvi.

Fai notare una cosa sola: sotto la parola sbagliata Python mette delle frecce `^^^^^^`. Indica esattamente dove.

Minuti 8–15: da soli

`fehler_2`, `fehler_3`, `fehler_4` sulle proprie macchine, in ordine.

> **La regola della caccia**
>
> Chi trova alza la mano e dice **solo il numero di riga**, non la soluzione. Così gli altri continuano a cercare e tu vedi subito chi è arrivato.

Minuti 15–20: i due senza messaggio

`fehler_5` e `fehler_6`, insieme al proiettore. Questi sono i più importanti: **nessun testo rosso, e il programma finisce normalmente.** Ma il disegno è sbagliato.

> »Kein roter Text. Und trotzdem falsch. Was sollte passieren — und was passiert?«

Qui la terza domanda cambia: non «cosa c'è di sbagliato», ma **«cosa manca»**.

Due cose da non dire

Non dire che gli spazi danno errore. `penup ()` con lo spazio funziona benissimo, Python se ne infischia. Se lo dici insegni una superstizione che poi ti si ritorce contro.

Non dire «hai sbagliato». Il messaggio rosso non è una punizione, è l'unico aiuto che hanno. Il modo in cui lo tratti tu decide se lo leggeranno o no per i prossimi tre anni.

Le soluzioni complete, coi messaggi veri di Python, sono nell'**Allegato A**.

Fase 3 · I file di esempio (20–25)

Cinque minuti, al proiettore, tre secondi a file: apri, Play, chiudi. Non spieghi niente, fai vedere e basta.

File	Cosa esce
`kreis.py`	un cerchio azzurro pieno
`quadrat.py`	un quadrato arancione
`dreieck.py`	un triangolo verde
`dreieck_frei.py`	un triangolo qualsiasi, fatto con tre punti
`bogen.py`	un arco di trionfo: due colonne, un arco, un architrave
`treppe.py`	una scala di cinque gradini
`stern.py`	una stella a cinque punte
`haus.py`	una casa col tetto
`hintergrund.py`	cielo, prato, sole — e lo spazio per il loro pezzo

Poi la regola d'uso, che è tutta la lezione:

> »Such dir was aus. Kopieren, in dein Bild einsetzen, Zahlen ändern. Nicht abtippen — **kopieren**.«

Copiare e incollare non è barare: è quello che fanno tutti i programmatori. Dillo chiaramente, perché qualcuno penserà di star imbrogliando.

Fase 4 · Il triangolo, due modi (25–30)

L'unico pezzo di teoria di oggi, e c'è perché il triangolo è quello che li frega sempre.

Modo 1 · tutti i lati uguali

```
forward(150)
left(120)
forward(150)
left(120)
forward(150)
left(120)
```

Funziona **solo** se i tre numeri sono uguali. La rotazione è sempre 120, perché 360 diviso 3 fa 120.

> »Wenn die drei Zahlen nicht gleich sind, geht das Dreieck nie zu. Probiert es — macht eine davon 80.«

Falli provare davvero, trenta secondi: è l'unico modo perché se lo ricordino.

Modo 2 · tre punti

Se vogliono un triangolo diverso — più largo, più piatto, storto — gli angoli non bastano: dovrebbero calcolare anche la lunghezza dei lati, e non è roba da oggi.

La via d'uscita è non usare gli angoli affatto:

```
penup()
goto(-100, -80)       # Ecke 1
pendown()
begin_fill()
goto(100, -80)        # Ecke 2
goto(0, 90)           # die Spitze
goto(-100, -80)       # zurück zu Ecke 1
end_fill()
```

> »Du sagst nur, wo die drei Ecken sind. Die Längen rechnet der Computer.«

L'ultima riga — tornare al primo punto — è quella che dimenticano tutti, e senza quella il triangolo resta aperto.

Quale dei due usare: il primo se il triangolo deve essere regolare (il tetto di una casa, un albero). Il secondo se deve avere

una forma precisa. Dillo così e non aggiungere altro.

Fase 5 · Lo sfondo (30–35)

Apri `hintergrund.py` al proiettore e scorri fino in fondo. In cima c'è lo sfondo già fatto — »nicht ändern« — e in fondo c'è questo:

```
# ============================================================
#  AB HIER BIST DU DRAN
# ============================================================

# --- Mein Bild (_____________) ---
penup()
goto(-200, -125)      # <-- wo dein Bild anfängt
pendown()
setheading(0)

# hier kommen deine Befehle
```

Tre cose da dire, una volta ciascuna:

1. **Si salva subito col proprio nome.** `Datei → Speichern unter`, `mia_bild.py`. Non si lavora sul file originale, se no la settimana prossima non ce l'ha più nessuno.
2. **Il nome va nell'intestazione**, nello spazio vuoto.
3. **Il `goto` in cima decide dove finisce il disegno.** Quello si cambia; il resto dello sfondo no.

> **Il montaggio lo fai tu, dopo**
>
> Non provare a far unire i file a loro: l'hai già visto giovedì scorso, costa più tempo di quanto ne valga. Raccogli i file con la chiavetta a fine ora e **unisci due o tre quadri per volta a casa**, in cinque minuti. La settimana dopo li proietti all'inizio: è un regalo che costa zero tempo di lezione e fa un ottimo effetto.

Fase 6 · Officina (35–80)

Quarantacinque minuti, ed è la fase più lunga che abbiano mai avuto. La consegna è una frase:

> »Bau dein Bild. Nimm, was du brauchst, aus den Dateien.«

Regole già note, da ripetere una volta sola:

- **Ein Block, dann Play.** Tre o quattro righe, Play, guarda.
- **Erst den Nachbarn fragen, dann mich.**
- Ogni pezzo nuovo vuole la sua intestazione con `#` e una riga vuota prima.

Cosa fai tu per quarantacinque minuti

Giri. Ma oggi hai un compito preciso e non è spiegare: quando ti chiamano per un errore, **non lo risolvi**. Punti il dito sul messaggio rosso e fai le tre domande della lavagna. Ci metti il doppio le prime cinque volte e un quarto dalla decima in poi.

Dal minuto 72 giri con la chiavetta e raccogli i file.

Fase 7 · Galleria (80–88)

Tre o quattro schermi sul proiettore, non di più. Scegli **uno dei lenti per primo**: oggi hanno un quadro vero anche loro, con cielo, prato e sole, e si vede.

Una domanda sola per ogni quadro, rivolta alla classe e non all'autore:

> »Welches Stück hat er genommen, und was hat er geändert?«

Fase 8 · Quaderno (88–90)

Disegnano il proprio quadro, una riga sotto: quale file hanno copiato e cosa hanno cambiato. Quaderni nella scatola.

Se finiscono prima

Se finiscono in due o tre, mentre gli altri lavorano

Un secondo pezzo, in un altro punto dello sfondo. Secondo blocco, seconda intestazione, secondo `goto`. Senza toccare il primo.

L'arco di trionfo rimpicciolito. Prendono `bogen.py` e lo fanno entrare in metà spazio: devono cambiare tutti i numeri in modo coerente, e scoprono da soli che dimezzare solo alcuni rompe tutto. È la variabile con tre settimane di anticipo.

Riparare un file rotto scritto da te. Tieni pronti due file rotti in più — stessi errori, disegni diversi — per chi finisce presto. La caccia all'errore piace più del previsto.

Se finiscono quasi tutti, e restano dieci o quindici minuti

La caccia a squadre. Proietti un file rotto che non hanno mai visto e la classe ha due minuti per trovare la riga. Chi la dice per primo ne proietta un altro.

Il quadro collettivo, questa volta fatto da te. Monti in diretta tre dei loro quadri sullo stesso sfondo, con loro che guardano, e commenti ad alta voce quello che fai: »jetzt kopiere ich Lisas Block hierher, und ihr Haus steht bei goto minus zweihundert.« Durano otto minuti e vedono come si fa, senza doverlo fare.

Se avanzano cinque minuti

Cambia un colore solo, e prima di premere Play dici al vicino cosa succederà.

Gli inciampi

Cosa vedi	Cos'è	Cosa fare
Testo rosso e mano alzata	L'abitudine vecchia	Non guardare lo schermo. Indica la lavagna: »welche Zeile?«
Niente rosso, «non va»	Parentesi o `end_fill()` dimenticati	»Was sollte passieren, und was passiert?«
Il triangolo non si chiude	Tre numeri diversi, o manca il ritorno al primo punto	Modo 1 o modo 2: fagli dire quale dei due sta usando
Ha cambiato lo sfondo e si è rotto tutto	Succede	`Datei → Rückgängig`, oppure ricopia `hintergrund.py` dalla cartella
Ha salvato sopra `hintergrund.py`	Succede di sicuro, almeno una volta	Per questo il file originale sta nella cartella e si lavora su una copia. Riporta il file dalla chiavetta
Il disegno finisce sotto il prato	Il `goto` in cima ha una y troppo bassa	Il prato arriva a −125. Sopra quello si vede, sotto no
Copia un pezzo e sparisce il colore di prima	`color()` del pezzo nuovo vale da lì in poi	Non è un errore: è come funziona. Se vuole il colore di prima lo riscrive

---

Differenziazione

Chi è lentissimo a scrivere. Oggi la lezione è fatta per lui: copia `haus.py` dentro lo sfondo, cambia due colori, e ha un quadro. Alla galleria il suo è il primo che proietti.

Chi ha voglia di lavorare. Non dargli più pezzi: dagli un vincolo. »Dein Bild muss aus mindestens vier Teilen bestehen, und keine zwei dürfen die gleiche Farbe haben.«

Chi si blocca e non chiama. Il giro dietro ai banchi ogni dieci minuti. Oggi passi comunque da tutti con la chiavetta, e lì vedi a che punto sono senza che nessuno alzi la mano.

Ha funzionato?

- Almeno sei ragazzi su undici hanno trovato da soli la riga di un errore.
- Nessuno, nella seconda metà dell'ora, ti ha chiamato senza prima aver letto il numero di riga.
- Tutti e undici hanno un file salvato col proprio nome e un quadro sullo sfondo.

- Qualcuno ha detto che le cinque scale di `treppe.py` sono troppe. **Segnati il nome.**
- Nel quaderno c'è scritto quale file hanno copiato.

Allegato A · Le soluzioni della Fehlerjagd

I messaggi qui sotto sono quelli veri, presi eseguendo i file. Thonny 5 usa Python 3.14 e la formulazione può cambiare di una parola — il numero di riga no.

	Errore	Cosa dice Python
`fehler_1.py`	il nome scritto male	`line 6 · NameError: name 'forwrd' is not defined. Did you mean: 'forward'?`
`fehler_2.py`	le virgolette mancanti	`line 5 · NameError: name 'red' is not defined`
`fehler_3.py`	la parentesi non chiusa	`line 6 · SyntaxError: '(' was never closed`
`fehler_4.py`	la maiuscola	`line 7 · NameError: name 'Left' is not defined. Did you mean: 'left'?`
`fehler_5.py`	le parentesi dimenticate	**nessun messaggio**
`fehler_6.py`	`end_fill()` dimenticato	**nessun messaggio**

`fehler_1.py` · il nome scritto male

Python scrive perfino il suggerimento, e le frecce `^^^^^^` indicano la parola esatta.

```
        # FEHLER 1 – Findest du ihn?
        from turtle import *
        speed(0)
        pensize(5)
        color("blue")
        forwrd(150)
        left(90)
        forward(150)
        done()
```

`fehler_2.py` · le virgolette mancanti

Senza virgolette `red` non è un colore: è un nome che Python cerca e non trova. È l'errore più frequente di tutti.

```
        # FEHLER 2 – Findest du ihn?
        from turtle import *
        speed(0)
        pensize(5)
        color(red)
        forward(150)
        left(90)
        forward(150)
        done()
```

`fehler_3.py` · la parentesi non chiusa

Con questo errore il programma non parte affatto. La riga indicata a volte è quella sopra il problema: regola generale, guarda quella riga **e quella prima**.

```
        # FEHLER 3 – Findest du ihn?
        from turtle import *
        speed(0)
        pensize(5)
        color("green")
        forward(150
        left(90)
        forward(150)
        done()
```

`fehler_4.py` · la maiuscola

Python distingue maiuscole e minuscole. Tutti i comandi della tartaruga sono minuscoli.

```
# FEHLER 4 — Findest du ihn?
from turtle import *
speed(0)
pensize(5)
color("purple")
forward(150)
Left(90)
forward(150)
done()
```

`fehler_5.py` · le parentesi dimenticate

`penup` senza parentesi non è un ordine: è solo il nome del comando. La penna non si alza mai e invece di due trattini separati esce una riga unica. Come dire «la porta» invece di «apri la porta».

```
# FEHLER 5 — Kein roter Text. Trotzdem ist es falsch.
# Es sollen ZWEI getrennte Striche sein, ohne Linie dazwischen.
from turtle import *
speed(0)
pensize(5)
color("orange")
forward(100)
penup
forward(60)
pendown
forward(100)
done()
```

`fehler_6.py` · `end_fill()` dimenticato

Il quadrato esce vuoto invece che dorato: il riempimento era cominciato e non è mai finito.

```
# FEHLER 6 — Kein roter Text. Trotzdem ist es falsch.
# Das Quadrat soll AUSGEMALT sein.
from turtle import *
speed(0)
color("black", "gold")
begin_fill()
forward(120)
left(90)
forward(120)
left(90)
forward(120)
left(90)
forward(120)
left(90)
done()
```

Allegato B · I file di esempio

Vanno copiati nella cartella `python_kurs` di ogni macchina il mercoledì. I sei file rotti stanno in una sottocartella `fehler`.

`hintergrund.py`

```
# ============================================================
#  HINTERGRUND – Himmel, Wiese und Sonne
#
#  Das hier ist schon fertig. Nicht ändern.
#  DEIN BILD KOMMT GANZ UNTEN, wo es steht.
# ============================================================

from turtle import *

setup(900, 650)
bgcolor("skyblue")
speed(0)
hideturtle()
pensize(3)

# --- Die Wiese ---
penup()
goto(-450, -325)
pendown()
setheading(0)
color("darkgreen", "yellowgreen")
begin_fill()
forward(900)
left(90)
forward(200)
left(90)
forward(900)
left(90)
forward(200)
left(90)
end_fill()

# --- Die Sonne ---
penup()
goto(300, 180)
pendown()
setheading(0)
color("orange", "gold")
begin_fill()
circle(45)
end_fill()


# ============================================================
#  AB HIER BIST DU DRAN
#  Schreib deinen Namen in die Überschrift.
# ============================================================

# --- Mein Bild (_____________) ---
penup()
goto(-200, -125)      # <-- wo dein Bild anfängt
pendown()
setheading(0)

# hier kommen deine Befehle




done()
```

kreis.py

```
# ============================================================
#  KREIS – ein bunter Kreis
#
#  Das Programm läuft schon. Ändere die Zahlen,
#  drück Play und schau, was passiert.
# ============================================================

from turtle import *

speed(0)

# --- Der Kreis ---
color("blue", "skyblue")     # <-- Strichfarbe, Füllfarbe
begin_fill()
circle(100)                  # <-- wie groß
end_fill()

done()
```

quadrat.py

```
# ============================================================
#  QUADRAT – ein buntes Quadrat
#
#  Probier mal: alle vier Zahlen auf 200.
#  Und dann: eine davon auf 50.
# ============================================================

from turtle import *

speed(0)

# --- Das Quadrat ---
color("black", "orange")     # <-- Strichfarbe, Füllfarbe
begin_fill()
forward(150)                 # <-- die vier Seiten
left(90)
forward(150)
left(90)
forward(150)
left(90)
forward(150)
left(90)
end_fill()

done()
```

dreieck.py

```
# ============================================================
#  DREIECK – ein Dreieck mit drei gleichen Seiten
#
#  WICHTIG, sonst geht es nie zu:
#  Alle drei Seiten sind GLEICH LANG, und jedes Mal
#  dreht man sich um 120 nach links.  360 : 3 = 120
#
#  Wenn du ein anderes Dreieck willst – breiter, flacher,
#  schief – dann geht das so nicht. Dafür gibt es dreieck_frei.py
# ============================================================

from turtle import *

speed(0)

# --- Das Dreieck ---
color("darkgreen", "lightgreen")     # <-- Strichfarbe, Füllfarbe
begin_fill()
forward(150)                         # <-- Seite 1
left(120)
forward(150)                         # <-- Seite 2: GLEICHE Zahl
left(120)
forward(150)                         # <-- Seite 3: GLEICHE Zahl
left(120)
end_fill()

done()
```

dreieck_frei.py

```
# ============================================================
#  DREIECK FREI – ein Dreieck, das du selbst bestimmst
#
#  Hier gibt es KEINE Winkel und KEINE Seitenlängen.
#  Du sagst nur: hier, hier und hier sind die drei Ecken.
#  Den Rest rechnet der Computer.
#
#  Probier mal: ändere nur die erste Zahl von der Spitze.
# ============================================================

from turtle import *

speed(0)

# --- Die drei Ecken ---
color("darkblue", "lightblue")

penup()
goto(-100, -80)        # <-- Ecke 1: unten links
pendown()
begin_fill()
goto(100, -80)         # <-- Ecke 2: unten rechts
goto(0, 90)            # <-- Ecke 3: die Spitze
goto(-100, -80)        # wieder zur Ecke 1 – sonst ist es nicht zu
end_fill()

done()
```

bogen.py

```
# ============================================================
#  BOGEN – ein Tor mit zwei Säulen und einem Bogen
#
#  Vier Blöcke: linke Säule, rechte Säule, der Bogen,
#  und oben ein Balken.
#  Der Bogen ist ein halber Kreis: circle(80, 180)
# ============================================================

from turtle import *

speed(0)
pensize(3)

# --- Die linke Säule ---
penup()
goto(-130, -150)
pendown()
setheading(0)
color("dimgray", "lightgray")
begin_fill()
forward(50)
left(90)
forward(150)
left(90)
forward(50)
left(90)
forward(150)
left(90)
end_fill()

# --- Die rechte Säule ---
penup()
goto(80, -150)
pendown()
setheading(0)
color("dimgray", "lightgray")
begin_fill()
forward(50)
left(90)
forward(150)
left(90)
forward(50)
left(90)
forward(150)
left(90)
end_fill()

# --- Der Bogen ---
penup()
goto(80, 0)
pendown()
setheading(90)
color("dimgray")
pensize(10)
circle(80, 180)      # <-- halber Kreis, Radius 80

# --- Der Balken oben ---
penup()
goto(-150, 85)
pendown()
setheading(0)
pensize(3)
color("dimgray", "silver")
begin_fill()
forward(300)
left(90)
forward(40)
left(90)
forward(300)
left(90)
forward(40)
left(90)
end_fill()

done()
```

treppe.py

```
# ============================================================
#  TREPPE – eine Treppe mit fünf Stufen
#
#  Schau dir an, wie oft das Gleiche dasteht.
#  Merk dir das. Nächste Woche brauchen wir das.
# ============================================================

from turtle import *

speed(0)
pensize(5)
color("saddlebrown")

# --- Stufe 1 ---
forward(40)
left(90)
forward(40)
right(90)

# --- Stufe 2 ---
forward(40)
left(90)
forward(40)
right(90)

# --- Stufe 3 ---
forward(40)
left(90)
forward(40)
right(90)

# --- Stufe 4 ---
forward(40)
left(90)
forward(40)
right(90)

# --- Stufe 5 ---
forward(40)
left(90)
forward(40)
right(90)

done()
```

stern.py

```
# ============================================================
#  STERN – ein Stern mit fünf Zacken
#
#  Die 144 ist das Geheimnis. Probier auch mal 150 oder 140.
# ============================================================

from turtle import *

speed(0)

# --- Der Stern ---
color("darkred", "gold")     # <-- Strichfarbe, Füllfarbe
begin_fill()
forward(200)
right(144)                   # <-- das Geheimnis
forward(200)
right(144)
forward(200)
right(144)
forward(200)
right(144)
forward(200)
right(144)
end_fill()

done()
```

`haus.py`

```
# ============================================================
#  HAUS – ein Haus mit Dach
#
#  Zwei Blöcke: erst die Wände, dann das Dach.
#  Ändere die Farben. Und probier mal, das Haus größer zu machen:
#  dann musst du ALLE 120 ändern, nicht nur eine.
# ============================================================

from turtle import *

speed(0)

# --- Die Wände ---
color("black", "firebrick")
begin_fill()
forward(120)
left(90)
forward(120)
left(90)
forward(120)
left(90)
forward(120)
left(90)
end_fill()

# --- Nach oben, ohne zu zeichnen ---
penup()
left(90)
forward(120)
right(90)
pendown()

# --- Das Dach ---
color("black", "sienna")
begin_fill()
left(60)
forward(120)
right(120)
forward(120)
right(120)
forward(120)
end_fill()

done()
```